Tuning-Free Image Editing with Fidelity and Editability via Unified Latent Diffusion Model

#rsiv

В этой статье приводится аккумуляция и доработка подходов к улучшению dual-branch алгоритма редактирования изображений.

Постановка задачи:

Дана оригинальная картинка I, промпт P, которому она соответствует и промпт P^*, описывающий требуемые изменения в картинке. Задача, получить I^* – изображение, которое семантически (внешне) как можно более близко к I и как можно лучше соответствует P^*

Как видно из постановки задачи, у нас есть trade-off. Если при редактировании мы максимизируем семантическое соответсие оригинальной картинке, мы можем потрать соответстие новому промпту P^* (under-edited). И наоборот, если слишком обусловиться на P^*, мы можем слишком видоизменить оригинальную картинку P (over-edited).

Разные состояний этого баланса (недо и пере редактирования) можно понять и на примере (Figure 1)

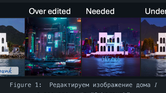

Figure 1: Редактируем изображение дома I с промптом: P: "Cyberpunk".
Примеры перередактирования, недоредактирования в сравнении с хорошим результатом

Краткое напоминание о структуре dual-branch подхода:

Вводные все те же I, P, P^*. Требуется получить I^*
Так же напомню, что в используемой conditioned unet есть два вида attention слоев:

- self-attention (между частями картинки)
- cross-attention (между частями картинки и промптом)

Алгоритм:
1. Используя инвертор (например Null-Text Inversion) переводим I в латентное представление z_0. Как правило это точка из $\mathcal{N}(0, I)$
2. Из латента z_0 параллельно запускаем две ветви денойза
   - Source Branch, обуславливающуюся на P и, восстанавливающую оригинальную картинку I из латента
   - Target Branch, обуславливающуюся на P^* и, генерирующую отредактированное (целевое) изображение I^*
3. На некоторых итерациях денойза делаем делаем инъекцию self-attention значений A^{self} из source ветки в соответствующие A^{self} target ветки, для того чтобы в I^* лучше сохранить внешний вид (семантику) оригинального I.

Схема dual-branch алгоритма изображена на Figure 2.

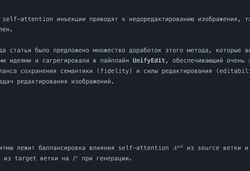

Figure 2: схема dual-branch алгоритма

Зачастую такие self-attention инъекции приводят к недоредактированию изображения, так что подход не идеален.

К моменту выхода статьи было предложено множество доработок этого метода, которые авторы дополнили своими идеями и загрегировали в пайплайн **UnifyEdit**, обеспечивающий очень хорошие результаты баланса сохранения семантики (fidelity) и силы редактирования (editability) во многих видах задач редактирования изображений.

UnifyEdit

В основе алгоритма лежит балансировка влияния self-attention A^{self} из source ветки и cross-attention A^{cross} из target ветки на I^* при генерации.

Наблюдения авторов, на которых строится алгоритм:

- если при генерации I^* мы приближаем $A^{self} \leftarrow A^{st}$, например инъекцией $A^{self} \leftarrow A^{st}$, то I^* лучше сохраняет семантику I. (fidelity)
- а если при генерации I^* мы усиливаем (позже опишу как) влияние A^{cross}, I^* точнее подходит под P^*. (editability)

UnifyEdit сторится на управлении этими двумя рычагами, один из которых позволяет сохранить семантику, I, а второй лучше подстроиться под промпт P^*.

Формализация UnifyEdit:

На каждом из шагов денойза t T, мы внесем корректировки в латент z_t таким образом, чтобы он лучше соответствовал нашим требованиям fidelity и editability.

А именно, мы сформируем функцию $\mathcal{L}$, такую что обновленный латент $z_t = z_t^* - \nabla_z \mathcal{L}$ соответствовал нашим требованиям лучше чем z_t^*. Авторы обощали свой метод на редактирование части изображения под маской $\mathcal{M} \in \{0, 1\}^{h \times w}$. Таким образом формула улучшения z_t^* приобретает вид

$$z_t = z_t^* - \mathcal{M} \odot \nabla_z \mathcal{L}$$

Для того, чтобы воплотить идею такого улучшения латента, $\mathcal{L}$ должна на шаге t численно выражать влияние A^{st} и A^{cross} на z_{t-1}.

В качестве влияния A^{st} вместо грубых инъекций предложили оптимизацию

$$\mathcal{L}_{SA} = sum_i (A_i^{self} - A_i^{st})^2$$

где i – номер self-attention слоя внутри Unet.
Для случая с $\mathcal{M}$ получается:

$$\mathcal{L}_{SA} = sum_i (\mathcal{M} \odot A_i^{st} - \mathcal{M} \odot A_i^{self})^2$$

В качестве влияния A^{cross} предложили

$$\mathcal{L}_{CLA} = -\left(\sum_i \sqrt{R_i}\right)^2, \text{где } R_i = \frac{\frac{1}{hw} \sum_{p \in \mathcal{M}} (A^{cross})_i^p}{\frac{1}{|\mathcal{M}|} \sum_{p \in \mathcal{M}} (A^{cross})_i^p}$$

Числитель R_i описывает среднее значение cross-attention между сектарами картинки $p \in \mathcal{M}$ и i-тым токеном промпта P^* на i-том cross-attention слое. Простыми словами – это то насколько сильно связан i-тый токен промпта с частью картинки под маской.

Знаменатель аналогично описывает как в среднем связан i-тый токен с картинкой вне маски.

Большое значение R_i говорит о том, что картинка в зоне редактирования сильнее соответствует целевому промпту P^*, чем вне этой зоны. $\mathcal{L}_{CLA}$ же просто агрегирует эти величины по всем cross-attention слоям в Unet.

И того, для задачи $z_t^* = z_t^* - \mathcal{M} \odot \nabla_z \mathcal{L}$ авторы подобрали $\mathcal{L} = \lambda_s \mathcal{L}_{SA} + \lambda_c \mathcal{L}_{CLA}$. И получили

$$\nabla_z \mathcal{L} = \mathcal{S}_{mm} = \lambda_1 \nabla_z \mathcal{L}_{SA} + \lambda_2 \nabla_z \mathcal{L}_{CLA}$$

Но $\mathcal{S}_{mm}$ плохо себя показала на практике.
Авторы выдвинули два оправдывающих себя на предположения:

1. Проблема в том, что масштабы $\nabla_z \mathcal{L}_{SA}$ и $\nabla_z \mathcal{L}_{CLA}$ разные и один из них ($\mathcal{L}_{SA}$) не достаточно влияет на улучшение z_t^*
2. Поскольку первые шаги разумнения определяет общие формы и цвета (fidelity), а последние – текстуры и не большие объекты (editability), мы хотим на первых шагах увеличить влияние $\mathcal{L}_{SA}$, а на последних $\mathcal{L}_{SA}$.

Таким образом, получилась финальная хорошая версия лосса, контролирующая, масштабы и предугадывающая изменение влияния $\mathcal{L}_{SA}, \mathcal{L}_{CLA}$ согласно интуиции появления семантики на шагах разумнения:

$$\nabla_z \mathcal{L} = \mathcal{S}_{ms} = \lambda_1^t \frac{\nabla_z \mathcal{L}_{SA}}{\|\nabla_z \mathcal{L}_{SA}\|_2} + \lambda_2^t \frac{\nabla_z \mathcal{L}_{CLA}}{\|\nabla_z \mathcal{L}_{CLA}\|_2}, \text{где } \begin{cases} \lambda_1^t = \beta_1 (1 - e^{-k_1(T-t)}) \\ \lambda_2^t = \beta_2 e^{-k_2(T-t)} \end{cases}$$

У нашего алгоритма появились параметры:

- β_1, β_2, контролирующие масштабы $\nabla_z \mathcal{L}_{SA}$ и $\nabla_z \mathcal{L}_{CLA}$ соответственно
- k_1, k_2, контролирующие скорость изменения влияния $\nabla_z \mathcal{L}_{SA}$ и $\nabla_z \mathcal{L}_{CLA}$ на итоговый лосс соответственно.

И последнее дополнение от авторов заключалось в том, что на каждом шаге разумнения можно делать не один шаг улучшения z_t^*, а вплоть до **MAX_IT** – последний гиперпараметр системы.
Таким образом итоговая схема шага диффузионной модели выглядит так:

$\lambda_1 = \beta_1 e^{-k_1 t}$;
$\lambda_2 = \beta_2 (1 - e^{-k_2 t})$;
for i = 1 **to** MAX_IT **do**
  $A_t^{self}, A_t^{st} \leftarrow SD(z_t, P, t)$;
  $A_t^{self}, A_t^{cross} \leftarrow SD(z_t^*, P^*, t)$;
  $\mathcal{L}_{SAP} \leftarrow \mathcal{F}_1(A_t^{self}, A_t^{self})$;
  $\mathcal{L}_{CLA} \leftarrow \mathcal{F}_2(A_t^{cross})$;
  $\nabla_z \mathcal{L}_{SAP}$
  $\mathcal{G}_{ms} = \lambda_1^t \frac{\nabla_z \mathcal{L}_{SAP}}{\|\nabla_z \mathcal{L}_{SAP}\|_2} + \lambda_2^t \frac{\nabla_z \mathcal{L}_{CLA}}{\|\nabla_z \mathcal{L}_{CLA}\|_2}$;
  $z_t^* = z_t^* - \mathcal{M} \odot \mathcal{G}_{ms}$;
  // UnifyEdit.
end
$z_{t-1}^* \leftarrow SD(z_t^*, P^*, t)$;
Return z_{t-1}^*

Figure 3: Полная схема UnifyEdit алгоритма,
Где β_1, β_2 – функции вычисляющие $\mathcal{L}_{SA}, \mathcal{L}_{CLA}$ соответственно,
SD – диффузионная модель (в данном случае Stable Diffusion)

Результаты

Основываясь на таком пайплайне семплинга, у авторов получилось выбить одни из лучших показателей по fidelity и editability среди конкурирующих алгоритмов для всех рассмотренных ими видов редактирования изображения (например замена объекта, изменение цвета, текстуры, черт лица).

Так же авторы загегрировали датасеты по редактированию изображений, дополнили своими картинками и аккумулированный dataset Unify-Bench опубликовали.

Мои мысли насчет статьи

В статье круто доработали dual-branch метод редактирования картинок, и выбили топ скоры, но все-таки их метод оставляет не решенной

- задачу подбора параметров $\beta_1, \beta_2, k_1, k_2$, MAX_ITER. В своих экспериментах они использовали какие-то подобранные значения, но понять с ходу, какие гиперпараметры нужны для конкретной задачи едва ли возможно, а подбор оптимальных параметров не кажется таким уж тривиальным. Вообщем, если при инференсе по перфекционистски требовать от модели получить из возможных результатов, подбор параметров будет гемороем
- проблему не идеальности самой диффузионной модели, которая может плохо реализовать требуемое в P, например, если P^* = "рука с шестью пальцами", "медведь на передних лапах"), потому что с точным количеством объектов или не типичными для приоров модели промптами, диффузионочки не справляется уже.

Как развить мысль:
Авторы в целом предлагает улучшать z_t^* градиентным спуском. Было бы интересно попробовать разные модификации спуска, например SAG, Momentum. Они проиграют нам шестьсот к хорошим оптимумам в оптимизации лосса модели, а вдруг и тут они себя хорошо покажут. Интересно.